

Cours 02 — Branches et fusion

Module 2/8 — Formation Git & GitHub

Objectifs

À la fin de ce module, vous saurez : - expliquer ce qu'est une branche — un simple **pointeur mobile** vers un commit — et à quoi sert HEAD ; - créer, lister, basculer et supprimer des branches avec `branch` et `switch` ; - fusionner deux branches avec `merge`, en distinguant la **fusion rapide** (*fast-forward*) du **commit de fusion** ; - reconnaître un **conflit**, lire ses marqueurs, et le résoudre calmement.

Théorie

Une branche n'est qu'un pointeur

C'est l'idée la plus importante du module, et elle est contre-intuitive au début : une branche n'est **pas** une copie de vos fichiers. C'est juste une **étiquette de 40 caractères** qui contient le hash d'un commit. Rien de plus.

Quand vous committez, l'étiquette de la branche courante **avance toute seule** vers le nouveau commit. C'est pour ça que créer une branche est instantané et gratuit : Git écrit un fichier de 41 octets, point. Là où d'autres outils dupliquent un dossier entier, Git déplace un pointeur.

```
C1 — C2 — C3          ← main (l'étiquette « main » pointe sur C3)
      ▲
      HEAD              ← HEAD pointe sur main : « je travaille ici »
```

HEAD est un second pointeur, particulier : il désigne **où vous êtes**. La plupart du temps, HEAD pointe sur une branche (`main`), et cette branche pointe sur un commit. Quand vous committez, les deux avancent ensemble.

Créer une branche, c'est dédoubler le chemin

Créez `feature/stop-loss` : une deuxième étiquette apparaît, sur le **même** commit que `main`. Rien n'a bougé, rien n'a été copié.

```
C1 — C2 — C3          ← main
      ▲
      └── feature/stop-loss (les deux pointent sur C3)
```

Basculez dessus (`git switch`), committez deux fois : c'est **cette** étiquette qui avance, `main` reste sur C3.

```
C1 — C2 — C3 — C4 — C5    ← feature/stop-loss ← HEAD
      |
      └── main              (toujours sur C3)
```

Vos fichiers sur le disque reflètent maintenant C5. Rebasculez sur `main` : Git remet le répertoire de travail dans l'état de C3, comme si C4 et C5 n'existaient pas — ils existent, mais sur l'autre branche.

C'est ça, une branche : une ligne de développement isolée.

Les deux types de fusion

Fusionner (`git merge`), c'est ramener le travail d'une branche dans une autre. Git a deux façons de faire, et il choisit tout seul selon la forme de l'historique.

Type	Quand Git le choisit	Résultat
Fusion rapide (<i>fast-forward</i>)	La branche cible n'a pas bougé depuis la séparation	L'étiquette avance, aucun commit créé
Commit de fusion	Les deux branches ont divergé (chacune a de nouveaux commits)	Un commit spécial, à deux parents , est créé

Fusion rapide. Si `main` est resté sur C3 pendant que `feature` avançait, fusionner revient à faire glisser l'étiquette `main` jusqu'à C5. Il n'y a rien à réconcilier : le chemin est droit.

```
avant :  C1 — C2 — C3 — C4 — C5   ← feature
                ▲
                main
```

après : C1 — C2 — C3 — C4 — C5 ← feature, main (main a juste « rattrapé »)

Commit de fusion. Si `main` a lui aussi avancé (un commit C6) pendant que `feature` travaillait, les deux histoires ont divergé. Git ne peut plus se contenter de glisser : il crée un commit C7 qui a **deux parents** (C5 et C6), et qui combine les deux travaux.

```
C1 — C2 — C3 — C4 — C5 — C7   ← main (commit de fusion, 2 parents)
                |               ^
                |               |
                C6              (ce que main avait avancé de son côté)
```

Au fait : `switch`, `checkout`, `restore` ?

Vous verrez beaucoup de tutoriels utiliser `git checkout` pour changer de branche. Ce n'est pas faux, mais `checkout` était une commande **fourre-tout** : elle changeait de branche *et* restaurait des fichiers *et* détachait HEAD. Trop de rôles pour un seul verbe. Git a donc scindé ses usages en deux commandes claires (depuis 2.23) :

- `git switch` — changer de branche, *rien que ça* ;
- `git restore` — restaurer des fichiers (annuler des mods, dé-indexer).

Cette formation utilise `switch` et `restore`. Si vous croisez `git checkout <branche>` ailleurs, sachez que c'est l'équivalent de `git switch <branche>` — même effet, ancienne syntaxe.

Les conflits : Git vous passe le relais

Git fusionne tout seul tant que les deux branches ont touché des **zones différentes**. Un conflit survient que si les **mêmes lignes** ont été modifiées différemment de part et d'autre : là, Git ne peut pas deviner quelle version garder, alors il s'arrête et vous passe la main. Il insère des **marqueurs** dans le fichier concerné :

```
<<<<<< HEAD
prix = arrondir(prix, 2)
=====
prix = round(prix, 4)
>>>>>> feature/precision
```

Ça se lit ainsi : - entre `<<<<<< HEAD` et `=====` : **votre** version (la branche courante) ; - entre `=====` et `>>>>>> feature/precision` : la version **qui arrive** (la branche fusionnée).

Vous éditez le fichier pour ne garder que ce qui doit rester (une des deux, ou un mélange), vous **supprimez les trois marqueurs**, puis `git add` (« c'est résolu ») et `git commit` (qui conclut la fusion). Un conflit n'est pas une erreur : c'est Git qui vous demande d'arbitrer un choix qu'il n'a pas le droit de faire à votre place.

Commandes commentées

Lister et créer — `git branch`

```
git branch                # lister les branches ; * marque la branche
courante
git branch feature/stop-loss  # créer une branche (sans y basculer)
```

git branch seul est un affichage, il ne modifie rien. Avec un nom, il **crée** une étiquette sur le commit courant — mais vous restez où vous êtes. D’où l’intérêt de la commande suivante.

Basculer — git switch

```
git switch feature/stop-loss      # aller travailler sur cette branche
git switch -c feature/rapports    # créer ET basculer, en une seule commande (-c =
create)
git switch main                   # revenir sur main
git switch -                      # revenir à la branche précédente (comme cd -)
```

switch déplace HEAD et remet vos fichiers dans l’état de la branche visée. En pratique, on utilise presque toujours git switch -c <nom> : on crée la branche et on y saute d’un coup.

Fusionner — git merge

```
git switch main                  # on se place SUR la branche qui doit RECEVOIR
git merge feature/stop-loss      # on y fusionne le travail de l'autre branche
```

Règle à retenir : **on fusionne toujours depuis la branche de destination**. On se met sur main, puis on « tire » feature dedans. Git choisit seul entre fusion rapide et commit de fusion. Pour forcer un commit de fusion même quand une fusion rapide serait possible (utile pour garder une trace visible de la fonctionnalité) :

```
git merge --no-ff feature/stop-loss  # toujours créer un commit de fusion
```

Résoudre un conflit

```
git merge feature/precision
# → CONFLICT (content): Merge conflict in gestionnaire_ordres.py
```

```
git status                      # liste les fichiers « both modified » (en
conflit)
# ... vous éditez gestionnaire_ordres.py : choisir la bonne version,
#   retirer les marqueurs <<<<<< ===== >>>>>> ...
git add gestionnaire_ordres.py   # marquer ce fichier comme résolu
git commit                      # conclure la fusion (Git propose déjà un
message)
```

```
git merge --abort               # OU : tout annuler et revenir à l'état d'avant
la fusion
```

git merge --abort est votre bouton « annuler » : tant que vous n’avez pas committé, il remet le dépôt exactement dans l’état d’avant, comme si vous n’aviez jamais lancé la fusion.

Supprimer une branche — git branch -d

```
git branch -d feature/stop-loss   # supprimer une branche DÉJÀ fusionnée (sûr)
git branch -D feature/abandonnee  # FORCER la suppression d'une branche non
fusionnée
```

-d (minuscule) refuse de supprimer une branche dont le travail n’a pas été fusionné : c’est un garde-fou contre la perte de commits. -D (majuscule) passe outre — à n’utiliser que si vous assumez d’abandonner ce travail. Supprimer une branche n’efface que **l’étiquette**, pas les commits eux-mêmes (voir git reflog, cours 05).

Visualiser — git log --graph

```
git log --oneline --graph --all    # dessine le graphe de TOUTES les branches, en
ASCII
```

--graph dessine les embranchements et les fusions ; --all montre toutes les branches, pas seulement la courante. C’est la commande à réflexe pour « voir » où en est le projet.

Exercices pratiques

Reprenez le dépôt demo-git du cours 01 (ou créez-en un neuf avec un fichier quelconque).

1. **Fusion rapide.** Depuis main, créez feature/a, faites-y deux commits, revenez sur main et fusionnez. Vérifiez avec `git log --oneline --graph` qu'**aucun** commit de fusion n'a été créé (le chemin est droit).
2. **Commit de fusion.** Recommencez avec feature/b, mais faites **aussi** un commit sur main avant de fusionner. Observez cette fois le commit de fusion à deux parents dans le graphe.
3. **Conflit volontaire.** Sur deux branches, modifiez **la même ligne** d'un même fichier de deux façons différentes, fusionnez, puis résolvez le conflit à la main.
4. **Nettoyage.** Listez vos branches et supprimez avec `-d` toutes celles qui sont fusionnées.

Solutions des exercices

Solution 1 — Fusion rapide

```
git switch -c feature/a          # créer + basculer
echo "ligne 1" >> notes.md && git add . && git commit -m "feat: ligne 1"
echo "ligne 2" >> notes.md && git add . && git commit -m "feat: ligne 2"

git switch main                  # main est resté en arrière, il n'a pas bougé
git merge feature/a              # → « Fast-forward » : main rattrape simplement
feature/a
```

```
git log --oneline --graph
# * b2c3d4e feat: ligne 2
# * a1b2c3d feat: ligne 1
# ... ligne droite, PAS de commit « Merge » : c'est une fusion rapide.
```

Puisque main n'a rien fait de son côté, Git n'a eu qu'à faire glisser l'étiquette : aucun commit de fusion n'est nécessaire.

Solution 2 — Commit de fusion

```
git switch -c feature/b
echo "côté B" >> notes.md && git add . && git commit -m "feat: travail sur B"

git switch main
echo "côté main" >> autre.md && git add . && git commit -m "feat: travail sur main"
# ↑ MAINTENANT les deux branches ont divergé : chacune a un commit que l'autre n'a pas

git merge feature/b              # Git ouvre un éditeur pour le message du commit
de fusion                        # (sauvegardez et fermez pour accepter le message
                                par défaut)

git log --oneline --graph
# *   e5f6a7b Merge branch 'feature/b'      ← le commit de fusion, à deux parents
# | \
# |  * c3d4e5f feat: travail sur B
# * | b2c3d4e feat: travail sur main
# | /
# * a1b2c3d commit de départ
```

Le `| \ ... | /` dans le graphe est la signature visuelle d'un commit de fusion : deux lignes qui se séparent puis se rejoignent.

Solution 3 — Conflit volontaire

```
# Partir d'un fichier avec une ligne connue
echo "arrondi = 2" > config.txt && git add . && git commit -m "config initiale"

git switch -c feature/precision
echo "arrondi = 4" > config.txt && git add . && git commit -m "feat: précision à 4"

git switch main
echo "arrondi = 0" > config.txt && git add . && git commit -m "feat: arrondi
entier"

git merge feature/precision      # → CONFLICT dans config.txt
git status                      # config.txt apparaît en « both modified »
```

Ouvrez config.txt : vous y voyez les marqueurs.

```
<<<<<< HEAD
arrondi = 0
=====
arrondi = 4
>>>>>> feature/precision
```

Éditez pour ne garder qu'une ligne (par ex. arrondi = 4) et **supprimez les trois marqueurs**. Puis :

```
git add config.txt              # « c'est résolu »
git commit                     # conclut la fusion
git log --oneline --graph      # le commit de fusion apparaît
```

Solution 4 — Nettoyage

```
git branch                      # lister ; * = branche courante (on ne peut pas
la supprimer)
git switch main                 # se placer sur main d'abord

git branch -d feature/a        # supprimées sans protester : elles sont
fusionnées
git branch -d feature/b
git branch -d feature/precision
```

```
git branch                     # ne reste que main
```

Si -d refuse (« not fully merged »), c'est que la branche contient du travail non fusionné : vérifiez avant de forcer avec -D.

Points clés à retenir

- Une branche = **un pointeur** vers un commit ; en créer une est instantané, alors faites-le systématiquement, une par fonctionnalité.
- Ne travaillez **jamais** directement sur main : `git switch -c feature/x`, puis fusionnez.
- Git choisit seul entre **fusion rapide** (chemin droit, aucun commit) et **commit de fusion** (branches divergentes, commit à deux parents).
- Un conflit n'est pas une erreur : c'est Git qui vous demande d'arbitrer. Éditez, retirez les marqueurs, `git add`, `git commit`.
- `git merge --abort` permet toujours de revenir en arrière tant que la fusion n'est pas conclue.